

# KUBERNETES HANDBOOK

A Beginner-Friendly Guide to Kubernetes Objects and Configuration

DEVOPS LEARNING GUIDE

# Contents


| | | |
|---|---|---|
| 1 | Introduction to Kubernetes Objects | 3 |
| 2 | Namespaces | 6 |
| 3 | Pods | 9 |
| 4 | ReplicaSets | 13 |
| 5 | Deployments | 16 |
| 6 | Rolling Updates in Deployments | 20 |
| 7 | DaemonSets | 24 |
| 8 | ConfigMaps | 27 |
| 9 | Secrets | 30 |
| 10 | Services | 34 |
| 11 | Ingress | 38 |
| 12 | Persistent Volumes (PV) and Persistent Volume Claims (PVC) | 42 |
| 13 | StorageClass | 46 |
| 14 | StatefulSets | 49 |
| 15 | Helm | 54 |
| 16 | Kustomize | 58 |
| 17 | Quick Comparison and Revision | 63 |
| 18 | Command Quick Reference | 65 |

CHAPTER 1

# Introduction to Kubernetes Objects

## 1.1 What is a Kubernetes Object?

Kubernetes is a system that runs and manages containers. You do not tell Kubernetes *how* to do every small job. Instead, you tell it **what you want**. For example: "I want 3 copies of my Nginx container running." Each thing you describe is called a **Kubernetes object**.

A Kubernetes object is a **record of your wish**. Kubernetes saves this record and then works hard to make your wish come true. Pods, Deployments, Services and Namespaces are all objects.

## 1.2 Why Do We Need Kubernetes Objects?

- Objects tell Kubernetes **what** you want (for example, 3 Pods).
- Kubernetes keeps checking that the real world matches your wish. If something breaks, it fixes it.
- Objects are saved as text files, so you can keep them in Git and reuse them.

## 1.3 What is a Manifest File?

A **manifest file** is a text file that describes one or more objects. It is the written form of your wish. Manifests are normally written in **YAML**.

## 1.4 Why Do We Use YAML?

YAML is a simple text format that uses **indentation** (spaces at the start of a line) to show structure. It is easy for humans to read. Two rules matter most:

- Use **spaces**, never the Tab key. Two spaces per level is the common style.
- A list item starts with a dash `-`. A key and value look like `name: my-pod`.

## 1.5 The Four Main Parts of a Manifest

| Field | Meaning | Simple explanation |
|---|---|---|
| **apiVersion** | Which version of the Kubernetes API to use | The "language version" used to describe the object. |
| **kind** | The type of object | What you want to create: Pod, Service, Deployment... |
| **metadata** | Data about the object | Its name, labels, and Namespace. |
| **spec** | The specification | The details of the wish: what you want it to look like. |

### What is status?

You never write `status` yourself. Kubernetes fills it in. It shows **what is really happening now**. Kubernetes compares `spec` (what you want) with `status` (what exists) and fixes any difference.

## 1.6 A First Example

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: my-pod
spec:
  containers:
    - name: nginx
      image: nginx:latest
```

| Line | Explanation |
|---|---|
| **apiVersion: v1** | Pods belong to the core API group, called `v1`. |
| **kind: Pod** | We want to create a Pod. A Pod is the smallest unit that runs containers. |
| **metadata:** | The section for information about the Pod. |
| **name: my-pod** | The name of the Pod. Names must be unique inside one Namespace. |
| **spec:** | The section that describes what the Pod should contain. |
| **containers:** | A list of containers inside the Pod. |
| **- name: nginx** | The first (and only) container is called `nginx`. The dash starts a list item. |
| **image: nginx:latest** | The container image to download. `nginx` is the image name and `latest` is the tag (version). |

## 1.7 How Does Kubernetes Understand a YAML File?

1. `kubectl` reads your YAML file and converts it to JSON.
2. kubectl sends it to the **API Server**, the front door of Kubernetes.
3. The API Server checks that the object is valid and saves it in **etcd** (the cluster database).
4. Other parts of Kubernetes (the scheduler, controllers and kubelet) notice the new object and do the work.

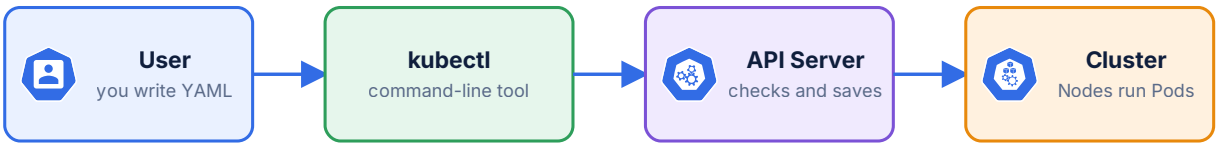



Diagram 1: How a manifest reaches the cluster

## 1.8 What Happens When We Apply a Manifest?

When you run `kubectl apply -f pod.yaml`, Kubernetes **compares** your file with what already exists. If the object does not exist, it creates it. If it exists, Kubernetes changes only what is different. This is called **declarative** management: you declare the end result, Kubernetes finds the steps.

## 1.9 Basic kubectl Commands

| Command | What it does |
| --- | --- |
| `kubectl apply -f filename.yaml` | Create or update objects from a file. |
| `kubectl get pods` | List Pods in the current Namespace. |
| `kubectl get all` | List the common objects (Pods, Services, Deployments...) in the current Namespace. |
| `kubectl describe pod pod-name` | Show full details and recent events of one Pod. Best command for troubleshooting. |
| `kubectl delete -f filename.yaml` | Delete the objects described in the file. |
| `kubectl get pods -o wide` | List Pods with extra columns, such as Pod IP and Node name. |
| `kubectl get pods -n namespace-name` | List Pods in a chosen Namespace. |

**COMMON MISTAKES & TROUBLESHOOTING**

- Using Tab characters or wrong spacing in YAML. Use spaces only.
- Writing the wrong `apiVersion` for a kind (for example, `v1` for a Deployment). Deployments use `apps/v1`.
- Forgetting `-f` in `kubectl apply`.
- Using the `latest` tag in real projects. It makes it hard to know which version is running.

CHAPTER 2

# Namespaces

## 2.1 What is a Namespace?

A **Namespace** is a named area inside a Kubernetes cluster. It groups objects together. Think of a cluster as a big office building and Namespaces as separate rooms. Different teams can work in different rooms.

> **NOTE**
> A Namespace gives **logical separation** inside one cluster. It is **not** a separate physical cluster. All Namespaces share the same Nodes and the same control plane.

## 2.2 Why Do We Need Namespaces?

- **Organise** objects: development, testing and production in one cluster.
- **Avoid name clashes**: two Namespaces can each have a Pod called `web`.
- **Control access and resources**: you can give a team permission to only one Namespace, and set limits on how much CPU or memory it may use.

## 2.3 Namespaces That Exist by Default

| Namespace | Purpose |
|---|---|
| **default** | Where your objects go if you do not choose a Namespace. |
| **kube-system** | Objects created by Kubernetes itself (DNS, network and proxy components). |
| **kube-public** | Readable by everyone, even without login. Rarely used for normal work. |
| **kube-node-lease** | Holds small "heartbeat" objects (Leases) that show each Node is alive. |

## 2.4 Creating a Namespace

`namespace.yaml`

```
apiVersion: v1
kind: Namespace
metadata:
  name: development
```

`kind: Namespace` creates a Namespace. `name: development` is its name. A Namespace needs no `spec` for basic use.

## 2.5 Creating Objects Inside a Namespace

Add `namespace` under `metadata`. Apply the Namespace file **first**, because the Namespace must exist before objects can be placed in it.

pod-dev.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  namespace: development
spec:
  containers:
    - name: nginx
      image: nginx:1.25
```

```
kubectl apply -f namespace.yaml
kubectl apply -f pod-dev.yaml
kubectl get pods -n development
```

## 2.6 Commands

| Command | What it does |
|---|---|
| `kubectl get namespaces` | List all Namespaces. |
| `kubectl create namespace development` | Create a Namespace quickly without a file. |
| `kubectl get pods -n development` | List Pods in the `development` Namespace. |
| `kubectl delete namespace development` | Delete the Namespace **and everything inside it**. |
| `kubectl config set-context --current --namespace=development` | Make `development` your default Namespace, so you can skip `-n`. |

**WARNING**

Deleting a Namespace deletes every object inside it. Always double-check the name before you press Enter.

## 2.7 Important Limitations

- Namespaces do **not** automatically block network traffic between them. Use NetworkPolicies for that.
- Some objects are **cluster-wide** and do not live in any Namespace: Nodes, PersistentVolumes, StorageClasses and Namespaces themselves.
- An object name must be unique only **inside** its Namespace.
- Namespaces cannot be nested inside each other.

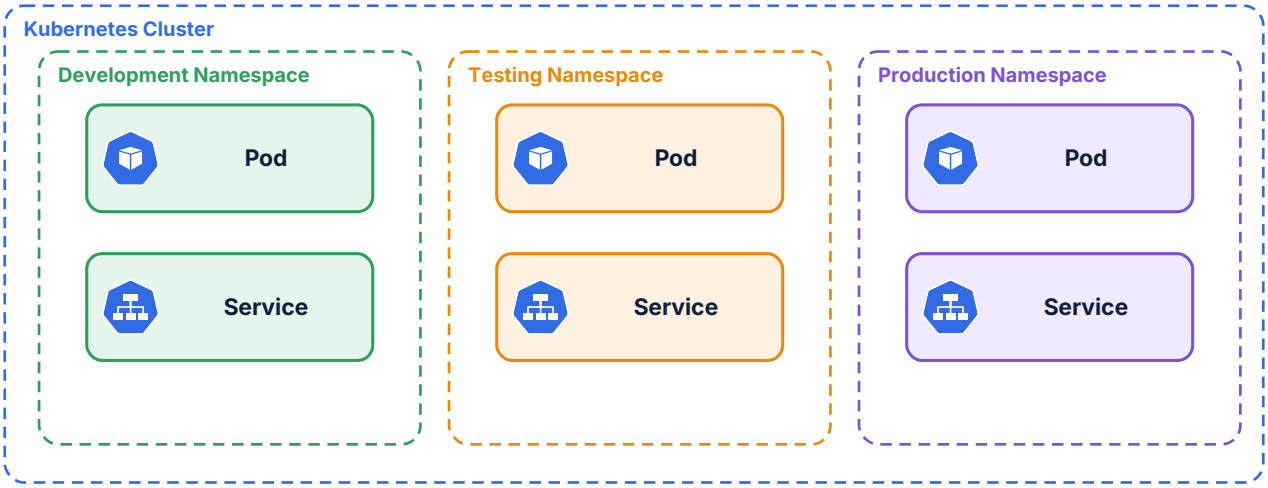



Diagram 2: One cluster with three Namespaces

**COMMON MISTAKES & TROUBLESHOOTING**

- Forgetting `-n` and wondering why a Pod is "missing". It is probably in another Namespace.
- Creating an object in a Namespace that does not exist yet.
- Thinking a Namespace gives full security isolation. It does not by itself.

CHAPTER 3

# Pods

## 3.1 What is a Pod?

A **Pod** is the smallest thing you can run in Kubernetes. A Pod wraps one or more containers. Kubernetes does not run a bare container; it always runs containers **inside a Pod**.

## 3.2 Why Do We Need Pods?

Some programs must work closely together, for example a web server and a helper that ships its logs. A Pod keeps them together: they start together, stop together, run on the same Node and share resources.

## 3.3 Docker Containers and Pods

With Docker you run `docker run nginx` and get a container. In Kubernetes, the container is placed inside a Pod. **Most Pods have just one container.** The Pod is like a small apartment, and the containers are the people living in it.

## 3.4 Multiple Containers in One Pod

Yes, a Pod can hold many containers. Containers in the same Pod:

- share the **same network and IP address**. They talk to each other using `localhost`.
- can share **storage** by mounting the same volume.
- are always placed on the **same Node**.

## 3.5 A Complete Nginx Pod

pod.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-pod
  labels:
    app: nginx
spec:
  containers:
    - name: nginx
      image: nginx:latest
      ports:
        - containerPort: 80
```

| Line | Explanation |
|---|---|
| **name: nginx-pod** | The Pod name. |
| **labels: app: nginx** | A label is a key-value tag. Services and ReplicaSets use labels to find Pods. |
| **containers:** | The list of containers in this Pod. |

| Line | Explanation |
| --- | --- |
| **image: nginx:latest** | The image Kubernetes downloads and runs. |
| **containerPort: 80** | Says the container listens on port 80. It is information for people and tools; it does not open the port to the outside. |

## 3.6 A Pod With Two Containers

Here a helper container writes a page into a shared volume, and Nginx serves it. The `emptyDir` volume is an empty folder that lives as long as the Pod lives.

two-container-pod.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: web-with-helper
  labels:
    app: web
spec:
  volumes:
    - name: shared-data
      emptyDir: {}
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
      volumeMounts:
        - name: shared-data
          mountPath: /usr/share/nginx/html
    - name: helper
      image: busybox:1.36
      command: ["sh", "-c", "while true; do date > /data/index.html; sleep 5; done"]
      volumeMounts:
        - name: shared-data
          mountPath: /data
```

`volumes` defines the shared folder. Each container uses `volumeMounts` to place it at a path. The helper writes the date to `/data/index.html`; Nginx sees the same file at `/usr/share/nginx/html/index.html`.

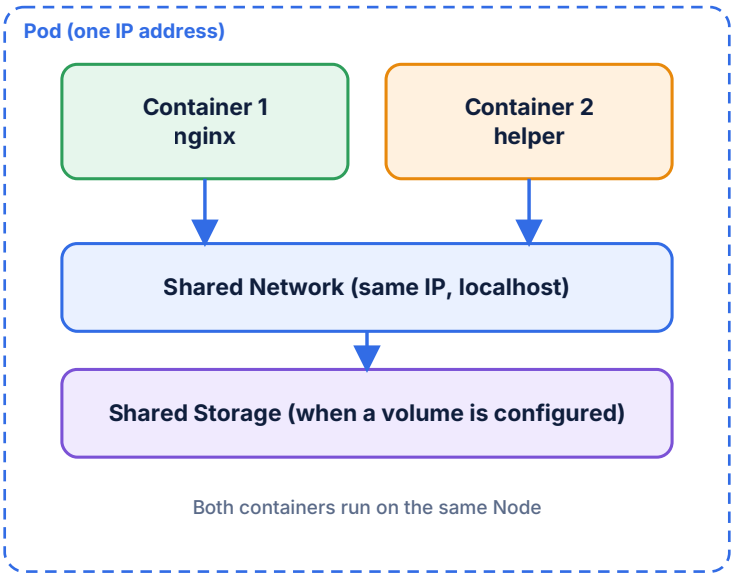



Diagram 3: A Pod with two containers

## 3.7 Pod Lifecycle and Phases

| Phase | Meaning |
|---|---|
| **Pending** | Pod is accepted, but containers are not running yet (waiting for a Node or for images to download). |
| **Running** | Pod is placed on a Node and at least one container is running or starting. |
| **Succeeded** | All containers finished successfully and will not restart (common for jobs). |
| **Failed** | All containers stopped and at least one ended with an error. |
| **Unknown** | Kubernetes cannot find out the state, usually because the Node cannot be reached. |

## 3.8 Restart Policies

Set `restartPolicy` under `spec`. It decides what happens when a container stops.

| Policy | Behaviour |
|---|---|
| **Always (default)** | Restart the container whenever it stops. |
| **OnFailure** | Restart only if it ended with an error. |
| **Never** | Never restart it. |

## 3.9 Pod IP Address, Labels and Resources

- **Pod IP:** every Pod gets its own IP. When a Pod is replaced, the new Pod gets a **different IP**.
- **Labels and selectors:** labels tag objects; a selector says "find all objects with this label". This is how Kubernetes connects objects.
- **Resource requests and limits:** a *request* is the minimum CPU/memory the container needs; a *limit* is the maximum it may use.

resources-snippet.yaml

```yaml
resources:
  requests:
    cpu: "100m"
    memory: "128Mi"
  limits:
    cpu: "250m"
    memory: "256Mi"
```

`100m` means 100 millicores, which is 0.1 of one CPU. `128Mi` means 128 mebibytes of memory. Put this block inside a container definition.

## 3.10 Pods Are Temporary

Pods are **replaceable**. If a Pod dies or its Node fails, a bare Pod is **not** brought back. Because of this, we normally do **not** create Pods directly in production. We use a **Deployment** (next chapters), which creates and replaces Pods for us.

## 3.11 Commands

| Command | What it does |
|---|---|
| `kubectl apply -f pod.yaml` | Create the Pod. |
| `kubectl get pods` | List Pods and their status. |
| `kubectl describe pod nginx-pod` | Show details and events. |
| `kubectl logs nginx-pod` | Show what the container printed. |
| `kubectl exec -it nginx-pod -- bash` | Open a shell inside the container. |
| `kubectl delete pod nginx-pod` | Delete the Pod. |
| `kubectl get pods -o wide` | Show Pod IP and Node. |

**Verify:** `kubectl get pods` should show `nginx-pod` with STATUS `Running` and READY `1/1`.

**Real-world example:** a web server container plus a log-shipping container in one Pod.

**COMMON MISTAKES & TROUBLESHOOTING**

- `ImagePullBackOff`: wrong image name or tag, or no access to the registry. Check with `kubectl describe pod`.
- `CrashLoopBackOff`: the container keeps crashing. Read `kubectl logs`.
- Pod stays `Pending`: not enough CPU/memory on Nodes. Check Events in `describe`.
- Labels differ between the Pod and its Service or ReplicaSet, so they never connect.
- Some images have no `bash`. Try `sh` instead.



CHAPTER 4

# ReplicaSets

## 4.1 What is a ReplicaSet?

A **ReplicaSet** makes sure that the required number of Pods are always running. If one Pod fails, Kubernetes creates another Pod automatically. A copy of a Pod is called a **replica**.

## 4.2 Why Do We Need ReplicaSets?

- A single Pod is not replaced when it dies. A ReplicaSet replaces it.
- Several replicas can share traffic, so your app stays available.
- You can increase or decrease the number of Pods easily.

## 4.3 How Does a ReplicaSet Work?

A ReplicaSet runs a simple loop: count the Pods that match its **selector**, compare with `replicas`, then create or delete Pods to match.

- **A Pod crashes or its Node fails:** the count drops, so a new Pod is created.
- **You delete a Pod by hand:** the ReplicaSet creates a new one immediately.
- **Too many Pods exist:** the ReplicaSet deletes the extra ones.

## 4.4 Labels and Selectors

The ReplicaSet does not "own" Pods by name. It finds them using a **label selector**. The labels in `selector.matchLabels` **must match** the labels in the Pod template.

## 4.5 A Complete ReplicaSet

replicaset.yaml

```yaml
apiVersion: apps/v1
kind: ReplicaSet
metadata:
  name: nginx-rs
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
```

| Field | Explanation |
| --- | --- |
| **apiVersion: apps/v1** | ReplicaSets belong to the `apps` API group. |
| **replicas: 3** | The desired number of Pods. |
| **selector.matchLabels** | How the ReplicaSet finds its Pods: it looks for label `app: nginx`. |
| **template** | The blueprint used to create new Pods. |
| **template.metadata.labels** | Labels given to each new Pod. **Must match the selector.** If not, Kubernetes rejects the ReplicaSet. |
| **template.spec** | The Pod specification (containers, image, ports). |

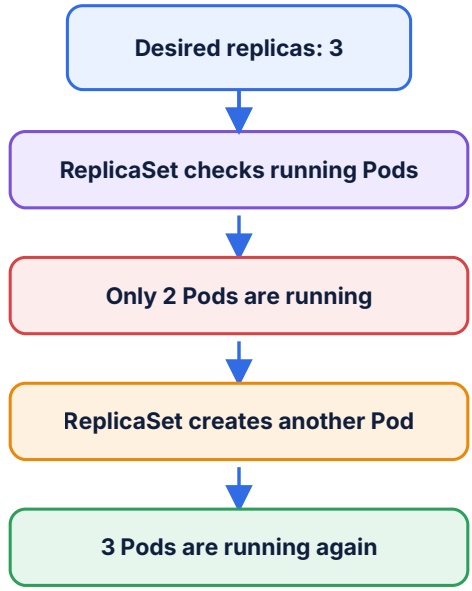



Diagram 4: A ReplicaSet restoring the desired number of Pods

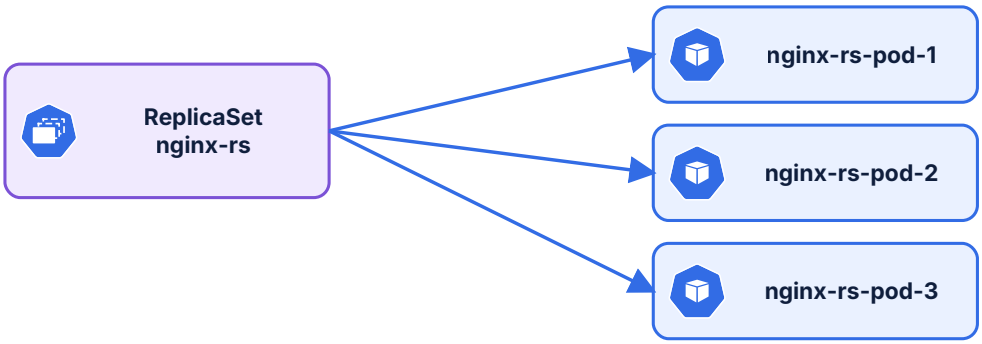



Diagram 5: ReplicaSet → 3 Pods

## 4.6 Apply, Verify and Use

```
kubectl apply -f replicaset.yaml
kubectl get replicasets
kubectl get pods --show-labels
```

| Command | What it does |
|---|---|
| `kubectl get replicasets` | List ReplicaSets with desired, current and ready counts. |
| `kubectl describe replicaset nginx-rs` | Show selector, template and events. |
| `kubectl get pods` | List the Pods it created. Names start with `nginx-rs-`. |
| `kubectl scale replicaset nginx-rs --replicas=5` | Change the number of Pods to 5. |

**Try it:** delete one Pod with `kubectl delete pod pod-name`, then run `kubectl get pods`. A new Pod appears within seconds.

## 4.7 Why Deployments Are Preferred

A ReplicaSet only keeps a number of Pods running. It cannot do smooth **updates** or **rollbacks**. A **Deployment** manages ReplicaSets for you and adds these features. So in real projects we create Deployments, not ReplicaSets.

**COMMON MISTAKES & TROUBLESHOOTING**

- Selector labels do not match template labels. The ReplicaSet is rejected.
- Creating a plain Pod with the same label ( `app: nginx` ). The ReplicaSet may count it as one of its Pods and delete extra ones.
- Editing the image in a ReplicaSet and expecting old Pods to update. Existing Pods are not changed. Use a Deployment.

CHAPTER 5

# Deployments

## 5.1 What is a Deployment?

A **Deployment** is the normal way to run an application in Kubernetes. You tell it which image to run and how many copies you want. It creates a ReplicaSet, and the ReplicaSet creates the Pods.

## 5.2 Why Do We Need Deployments?

- **Self-healing:** failed Pods are replaced.
- **Easy scaling:** change one number to run more Pods.
- **Safe updates:** change the image version without stopping the whole app.
- **Rollback:** go back to an older version if the new one is bad.

## 5.3 How a Deployment Works

A Deployment manages **ReplicaSets**, and each ReplicaSet manages **Pods**. When you change the Pod template (for example, the image), the Deployment creates a **new ReplicaSet** and slowly moves Pods from the old one to the new one. Old ReplicaSets are kept (with 0 Pods) so you can roll back.

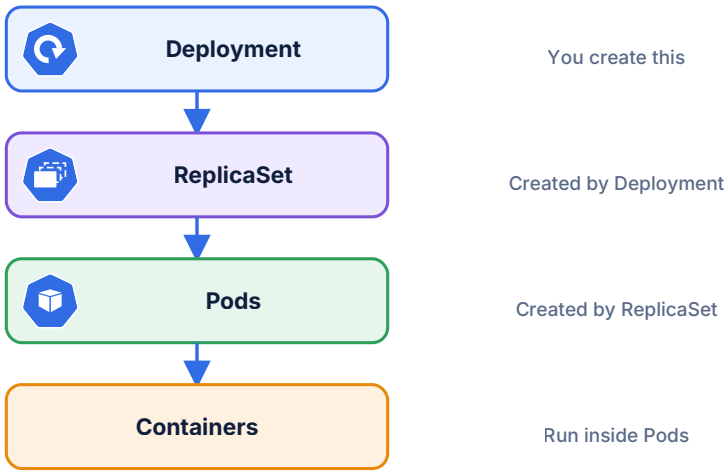



Diagram 6: Deployment → ReplicaSet → Pods → Containers

## 5.4 Key Ideas

| Idea | Simple meaning |
|---|---|
| **Desired state** | What you asked for (for example, 3 Pods with nginx:1.25). Kubernetes keeps working to match it. |
| **Declarative configuration** | You describe the end result in YAML. You do not write step-by-step instructions. |
| **replicas** | How many Pods you want. |

| Idea | Simple meaning |
| --- | --- |
| **Labels and selectors** | The selector tells the Deployment which Pods belong to it. It must match the Pod template labels. |
| **Scaling** | Changing `replicas` to run more or fewer Pods. |

## 5.5 Deployment Strategies

| Strategy | What it does | Result |
| --- | --- | --- |
| **RollingUpdate (default)** | Replaces Pods a few at a time. | The app stays available during the update. |
| **Recreate** | Stops **all** old Pods first, then starts new Pods. | Short downtime. Useful when two versions must never run together. |

## 5.6 A Complete Deployment

`deployment.yaml`

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
  labels:
    app: nginx
spec:
  replicas: 3
  selector:
    matchLabels:
      app: nginx
  strategy:
    type: RollingUpdate
    rollingUpdate:
      maxSurge: 1
      maxUnavailable: 1
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx:1.25
          ports:
            - containerPort: 80
          readinessProbe:
            httpGet:
              path: /
              port: 80
            initialDelaySeconds: 3
            periodSeconds: 5
```

| Field | Explanation |
| --- | --- |
| **apiVersion: apps/v1** | Deployments are in the `apps` API group. |
| **replicas: 3** | Run three Pods. |

| Field | Explanation |
| --- | --- |
| **selector.matchLabels** | Finds the Pods this Deployment owns. Must match `template.metadata.labels`. |
| **strategy.type** | `RollingUpdate` or `Recreate`. |
| **maxSurge: 1** | During an update, up to 1 extra Pod may exist above the desired count. |
| **maxUnavailable: 1** | During an update, up to 1 Pod may be unavailable. |
| **template** | The Pod blueprint. Changing anything here starts a new rollout. |
| **image: nginx:1.25** | The version to run. Changing it triggers a rolling update. |
| **readinessProbe** | A check that tells Kubernetes when a Pod is ready for traffic. Here: a request to `/` on port 80. |

## 5.7 Pause, Resume, History and Rollback

- **Pause** (`kubectl rollout pause`) lets you make several changes without starting a rollout each time. **Resume** then applies them together.
- **History** lists past revisions. **Rollback** (`kubectl rollout undo`) returns to the previous revision.
- **Updating the image** can be done by editing the YAML and running `kubectl apply`, or with `kubectl set image` (Chapter 6).

## 5.8 Pod vs ReplicaSet vs Deployment

| | Pod | ReplicaSet | Deployment |
| --- | --- | --- | --- |
| **Runs containers** | Yes | Through Pods | Through ReplicaSets |
| **Replaces failed Pods** | No | Yes | Yes |
| **Rolling updates** | No | No | Yes |
| **Rollback** | No | No | Yes |
| **Use in production** | Rarely | Rarely | Yes |

## 5.9 Apply and Verify

| Command | What it does |
| --- | --- |
| `kubectl apply -f deployment.yaml` | Create or update the Deployment. |
| `kubectl get deployments` | Show READY, UP-TO-DATE and AVAILABLE counts. |
| `kubectl describe deployment nginx-deployment` | Show strategy, events and conditions. |
| `kubectl get replicasets` | See the ReplicaSet the Deployment created. |
| `kubectl get pods` | See the Pods. Names look like `nginx-deployment-<hash>-<id>`. |
| `kubectl scale deployment nginx-deployment --replicas=5` | Run 5 Pods. |
| `kubectl delete deployment nginx-deployment` | Delete the Deployment, its ReplicaSets and Pods. |

**Real-world example:** the web front end of a shopping site runs as a Deployment with 5 replicas, so traffic is shared and updates cause no downtime.

**COMMON MISTAKES & TROUBLESHOOTING**

- Selector does not match template labels. The API rejects the Deployment.
- Changing the selector later. It is not allowed on `apps/v1` Deployments; delete and recreate instead.
- Scaling with `kubectl scale`, then applying an old YAML that still says the old replica count.
- Pods stuck `Pending`: Nodes have too little CPU or memory.

CHAPTER 6

# Rolling Updates in Deployments

## 6.1 What is a Rolling Update?

A **Rolling Update** replaces old Pods with new Pods **a few at a time**. At every moment, some Pods are still serving users.

## 6.2 Why Do We Need Rolling Updates?

Without them, you would stop the old version and start the new one, and users would see downtime. A rolling update gives **zero or very little downtime**.

## 6.3 How It Works Step by Step

1. You change the image (for example, `nginx:1.25` to `nginx:1.26`).
2. The Deployment creates a **new ReplicaSet** for the new version.
3. The new ReplicaSet starts creating new Pods. The old ReplicaSet removes old Pods.
4. A new Pod counts as **ready** only after its readiness check passes.
5. When all Pods are new, the old ReplicaSet has 0 Pods. It is kept for rollback.

## 6.4 maxSurge and maxUnavailable

| Setting | Meaning | Example with 3 replicas |
|---|---|---|
| **maxSurge** | How many **extra** Pods may be created above the desired number. | 1 means up to 4 Pods in total. |
| **maxUnavailable** | How many Pods may be **unavailable** during the update. | 1 means at least 2 Pods must stay available. |

Both can be a number or a percentage such as `25%`. They cannot both be 0.

strategy snippet

```
strategy:
  type: RollingUpdate
  rollingUpdate:
    maxSurge: 1
    maxUnavailable: 1
```




Diagram 7: Rolling update from nginx:1.25 to nginx:1.26 (simplified; exact order can vary)

## 6.5 Running the Update

| Command | What it does |
|---|---|
| `kubectl set image deployment/nginx-deployment nginx=nginx:1.26` | Change the image of container `nginx` to version 1.26. This starts the rolling update. |
| `kubectl rollout status deployment/nginx-deployment` | Watch the update. It waits until it succeeds or fails. |
| `kubectl rollout history deployment/nginx-deployment` | List the revisions. |
| `kubectl rollout history deployment/nginx-deployment --revision=1` | Show details of revision 1. |
| `kubectl rollout undo deployment/nginx-deployment` | Go back to the previous revision (manual rollback). |
| `kubectl rollout restart deployment/nginx-deployment` | Restart all Pods with a rolling update, without changing the image. |
| `kubectl describe deployment nginx-deployment` | See events, conditions and old/new ReplicaSets. |

**REMEMBER**

In `nginx=nginx:1.26`, the first `nginx` is the **container name** from the YAML, and the second part is the **image**. Revision history is more useful if you add the annotation `kubernetes.io/change-cause`.

## 6.6 What If the New Image Is Wrong?

- **ImagePullBackOff:** Kubernetes cannot download the image (wrong name or tag, or no permission). The new Pod never starts.
- **CrashLoopBackOff:** the container starts but keeps crashing. Kubernetes keeps restarting it with longer waits.
- **Not ready:** the container runs but fails its readiness check, so it gets no traffic and does not count as available.

In all three cases the **old Pods that are still running keep serving users**, so the update stalls instead of taking the whole app down. Using `maxUnavailable: 0` gives even more protection, because no old Pod is removed until a new one is ready.

## 6.7 How Kubernetes Checks Readiness

A **readinessProbe** tests a Pod regularly (for example, an HTTP request). Only Pods that pass are ready for traffic, and the rolling update only moves on when new Pods are ready. Without a readiness probe, Kubernetes considers a Pod ready as soon as its container starts, which can be too early.

> **WARNING**
> A normal Deployment does **not** automatically roll back a failed update. It stops making progress. After `progressDeadlineSeconds` (default 600 seconds) it marks the rollout as failed (`ProgressDeadlineExceeded`), but it leaves the broken Pods and old Pods as they are. A person, or automation, must run the rollback.

## 6.8 Automatic vs Manual Rollback

| | Manual rollback | Automatic rollback |
|---|---|---|
| **Who does it** | You run `kubectl rollout undo`. | A script, CI/CD pipeline or tool does it. |
| **Built into Deployment?** | The command is built in. | No. It needs extra automation. |
| **How it knows** | You notice the problem. | The pipeline checks `rollout status` and sees it failed. |

A CI/CD tool such as Jenkins can run `kubectl rollout status deployment/nginx-deployment --timeout=120s`. If that command fails (non-zero exit code), the pipeline runs:

```
kubectl rollout undo deployment/nginx-deployment
```

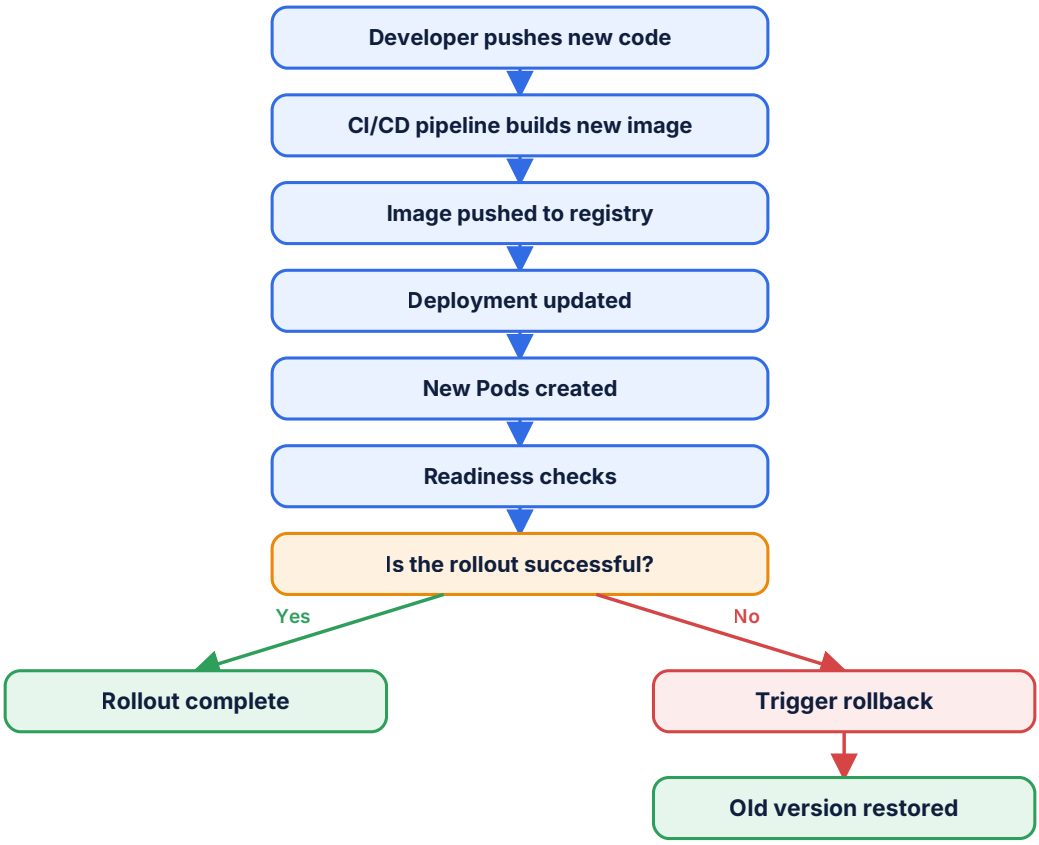



Diagram 8: A CI/CD rolling update with automatic rollback

**COMMON MISTAKES & TROUBLESHOOTING**

- No readiness probe, so a Pod that is not really working is counted as ready.
- Using `latest` as the image tag. Rolling updates only start when the Pod template changes.
- Forgetting the container name in `kubectl set image`.
- Expecting Kubernetes to roll back by itself. It does not.
- Rollback is not possible past `revisionHistoryLimit` (default 10 old ReplicaSets).



CHAPTER 7

# DaemonSets

## 7.1 What is a DaemonSet?

A **DaemonSet** makes sure that one copy of a Pod runs on **every eligible worker Node** (or on selected Nodes). You do not set a replica count; the number of Pods follows the number of Nodes.

## 7.2 Why Do We Need DaemonSets?

Some jobs must run on **every machine**, such as collecting logs, monitoring the Node, or setting up networking. A DaemonSet does this for you.

## 7.3 How It Works

- With 3 eligible Nodes you get 3 Pods, one per Node.
- When a **new Node joins**, the DaemonSet automatically starts a Pod on it.
- When a **Node is removed**, its Pod is deleted too.
- "Eligible" depends on scheduling rules: node selectors, affinity and taints/tolerations.

## 7.4 DaemonSet vs Deployment

| | Deployment | DaemonSet |
|---|---|---|
| **Number of Pods** | You choose with `replicas`. | One per eligible Node. |
| **Placement** | Scheduler picks any Node. | Every eligible Node. |
| **Typical use** | Web apps, APIs. | Log collectors, monitoring agents, network plugins. |

**Real-world examples:** Fluent Bit (log collection), Prometheus Node Exporter (Node metrics), and kube-proxy (networking) run as DaemonSets.

## 7.5 A Complete DaemonSet (Fluent Bit)

daemonset.yaml

```yaml
apiVersion: apps/v1
kind: DaemonSet
metadata:
  name: fluent-bit
  labels:
    app: fluent-bit
spec:
  selector:
    matchLabels:
      app: fluent-bit
  template:
    metadata:
      labels:
        app: fluent-bit
    spec:
      containers:
        - name: fluent-bit
          image: fluent/fluent-bit:2.2.2
          resources:
            limits:
              memory: 200Mi
            requests:
              cpu: 50m
              memory: 100Mi
          volumeMounts:
            - name: varlog
              mountPath: /var/log
              readOnly: true
      volumes:
        - name: varlog
          hostPath:
            path: /var/log
```

| Line | Explanation |
| --- | --- |
| **kind: DaemonSet** | Run one Pod per eligible Node. There is no `replicas` field. |
| **selector / labels** | Same rule as before: selector labels must match template labels. |
| **image: fluent/fluent-bit: 2.2.2** | A small log-processing tool. |
| **resources** | Requests and limits so the agent does not use too much of the Node. |
| **volumes: hostPath: /var/ log** | Gives the Pod access to the Node's log folder. This is a good use of hostPath: the agent must read files from the Node itself. |
| **volumeMounts (readOnly)** | Mounts that folder inside the container, read-only for safety. |

NOTE

This runs with Fluent Bit's default settings, which print sample data to the Pod logs. A real log setup needs a Fluent Bit configuration (often kept in a ConfigMap) that says where to send logs. Control-plane Nodes usually have a taint, so a Pod without a matching `toleration` will not run there.

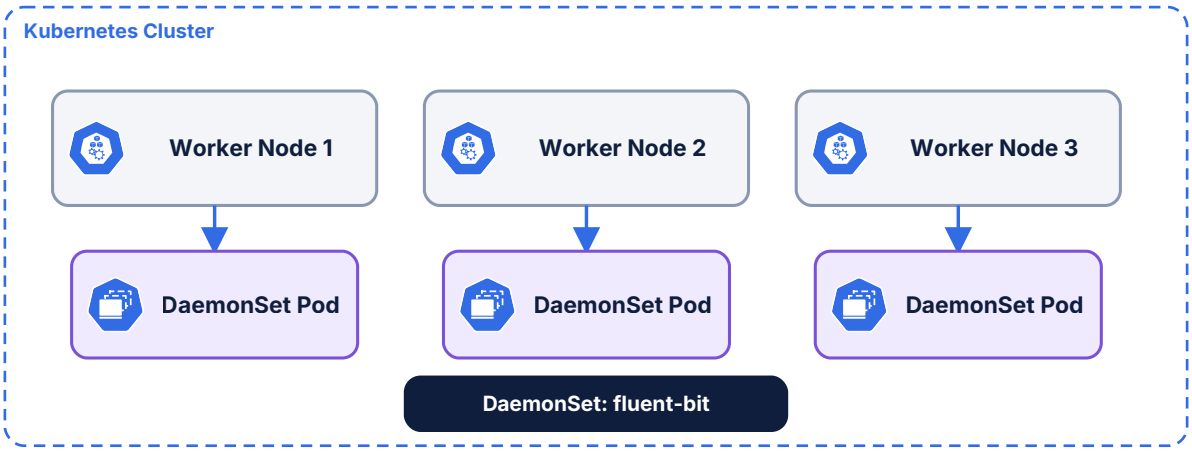



Diagram 9: A DaemonSet runs one Pod on each worker Node

| Command | What it does |
| --- | --- |
| `kubectl apply -f daemonset.yaml` | Create the DaemonSet. |
| `kubectl get daemonsets` | Show DESIRED, CURRENT and READY (one per Node). |
| `kubectl get pods -o wide` | See which Node each Pod runs on. |
| `kubectl describe daemonset fluent-bit` | Show details and events. |

**COMMON MISTAKES & TROUBLESHOOTING**

- Expecting a Pod on control-plane Nodes. They are usually tainted; add a toleration if needed.
- Forgetting resource limits, so the agent uses too much memory.
- Using a Deployment when you really need one Pod on every Node.

CHAPTER 8

# ConfigMaps

## 8.1 What is a ConfigMap?

A **ConfigMap** stores **non-secret configuration data** as key-value pairs, outside your container image. Pods can read it as environment variables, command arguments or files.

## 8.2 Why Do We Need ConfigMaps?

If you put configuration inside the image, you must rebuild the image for every change, and you need a different image for development and production. With ConfigMaps you use **one image** and change only the configuration.

## 8.3 What Can You Store?

- Small settings, like `APP_ENV=development` or `LOG_LEVEL=info`.
- Whole files, like an Nginx config or an HTML page.
- Limit: a ConfigMap can hold up to **1 MiB** of data.

> **WARNING**
> ConfigMaps are **not** for passwords, tokens or keys. Their data is stored as plain text. Use a Secret (Chapter 9) for sensitive data.

## 8.4 Example A: Simple Key-Value ConfigMap

configmap.yaml

```
apiVersion: v1
kind: ConfigMap
metadata:
  name: my-config
data:
  APP_ENV: development
  LOG_LEVEL: info
  index.html: |
    <h1>Hello from a ConfigMap</h1>
```

`data` holds the keys and values. The first two are simple settings. `index.html` is a key whose value is a whole file; the `|` means "keep the following lines exactly as written".

## 8.5 Example B: Use as Environment Variables

pod-env.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-env
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      envFrom:
        - configMapRef:
            name: my-config
      env:
        - name: MY_LOG_LEVEL
          valueFrom:
            configMapKeyRef:
              name: my-config
              key: LOG_LEVEL
```

- `envFrom` with `configMapRef` loads **all** keys as environment variables.
- `env` with `configMapKeyRef` loads **one key** and lets you choose the variable name ( `MY_LOG_LEVEL` ).

## 8.6 Example C: Mount as a Volume

pod-volume.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-volume
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
      volumeMounts:
        - name: config-volume
          mountPath: /usr/share/nginx/html
  volumes:
    - name: config-volume
      configMap:
        name: my-config
```

Each key becomes a **file** inside the mount folder. Nginx will serve `index.html` from the ConfigMap. Because the whole folder is replaced, `APP_ENV` and `LOG_LEVEL` also appear as files there.

**Apply order:** create the ConfigMap first, then the Pods.

## 8.7 Creating from Literals or Files

```
kubectl create configmap my-config --from-literal=APP_ENV=development
kubectl create configmap nginx-conf --from-file=nginx.conf
```

## 8.8 Updating ConfigMaps

- **Environment variables do not change** in running Pods. Restart the Pods to pick up new values (for example, `kubectl rollout restart`).
- **Mounted volumes update automatically**, but with a delay of up to about a minute. The application must re-read the file.
- ConfigMaps mounted with `subPath` do not update automatically.
- You can mark a ConfigMap `immutable: true` to prevent changes.

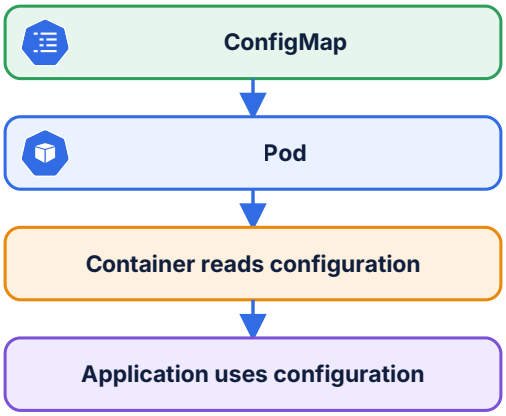



Diagram 10: ConfigMap and Pod

| Command | What it does |
|---|---|
| `kubectl get configmaps` | List ConfigMaps. |
| `kubectl describe configmap my-config` | Show the data inside. |
| `kubectl create configmap my-config --from-literal=APP_ENV=development` | Create one from the command line. |
| `kubectl delete configmap my-config` | Delete it. |

**Verify:** `kubectl exec nginx-env -- printenv APP_ENV` should print `development`.

## 8.9 Limitations

- Maximum 1 MiB per ConfigMap.
- Namespaced: a Pod can only use ConfigMaps from its own Namespace.
- Not for confidential data.
- Env values are read once, when the container starts.

**COMMON MISTAKES & TROUBLESHOOTING**

- Pod stays `ContainerCreating` or errors: the ConfigMap does not exist yet or has a different name.
- A wrong key name in `configMapKeyRef` prevents the container from starting.
- Mounting a ConfigMap over a folder hides the files already there.

CHAPTER 9

# Secrets

## 9.1 What is a Secret?

A **Secret** is an object that stores **sensitive data** such as passwords, API tokens and keys. Pods can use it as environment variables or as files.

## 9.2 Why Do We Need Secrets?

You should never write a password inside an image or a Pod YAML. A Secret keeps sensitive data in its own object, so you can control who can read it and keep it out of your images.

## 9.3 ConfigMap vs Secret

| | ConfigMap | Secret |
|---|---|---|
| **Stores** | Normal settings | Sensitive data |
| **Value format** | Plain text | Base64 text |
| **Special handling** | None | Can be encrypted at rest, restricted by RBAC, and is not shown in full by `kubectl describe` |
| **Size limit** | 1 MiB | 1 MiB |

## 9.4 Base64 is NOT Encryption

Secret values in the `data` field are stored in **Base64**. Base64 is just a way to write bytes using 64 safe characters. Anyone can turn it back into the original text in one command. It hides nothing.

```
echo -n 'admin' | base64          # prints YWRtaW4=
echo 'YWRtaW4=' | base64 -d       # prints admin
```

**WARNING**

Base64 is **encoding**, not **encryption**. Anyone who can read the Secret can read the password. Always use `-n` with `echo`, or a hidden newline gets encoded too.

## 9.5 Opaque Secret

The default Secret type is `Opaque`: you choose any keys and values.

secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
data:
  username: YWRtaW4=
  password: cGFzc3dvcmQ=
```

Here `YWRtaW4=` is `admin` and `cGFzc3dvcmQ=` is `password`.

### The easier way: stringData

With `stringData` you write plain text and Kubernetes encodes it for you when it stores the Secret.

secret-stringdata.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: db-secret
type: Opaque
stringData:
  username: admin
  password: password123
```

## 9.6 Other Secret Types

| Type | Used for |
| --- | --- |
| **Opaque** | General purpose key-value data. |
| **kubernetes.io/dockerconfigjson** | Login details for a private image registry. Created with `kubectl create secret docker-registry` and used through `imagePullSecrets` in a Pod. |
| **kubernetes.io/tls** | A TLS certificate and private key ( `tls.crt` and `tls.key` ). Used by Ingress for HTTPS. Created with `kubectl create secret tls`. |

## 9.7 Using a Secret in a Pod

pod-secret-env.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "echo user is $DB_USER && sleep 3600"]
      env:
        - name: DB_USER
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: username
        - name: DB_PASSWORD
          valueFrom:
            secretKeyRef:
              name: db-secret
              key: password
```

`secretKeyRef` reads one key from the Secret `db-secret` and gives it to the container as an environment variable. Create the Secret **before** the Pod.

### Mounting a Secret as a volume

pod-secret-volume.yaml

```
apiVersion: v1
kind: Pod
metadata:
  name: app-pod-vol
spec:
  containers:
    - name: app
      image: busybox:1.36
      command: ["sh", "-c", "sleep 3600"]
      volumeMounts:
        - name: secret-vol
          mountPath: /etc/db-secret
          readOnly: true
  volumes:
    - name: secret-vol
      secret:
        secretName: db-secret
```

Each key becomes a file: `/etc/db-secret/username` and `/etc/db-secret/password`. Files update when the Secret changes; environment variables do not.

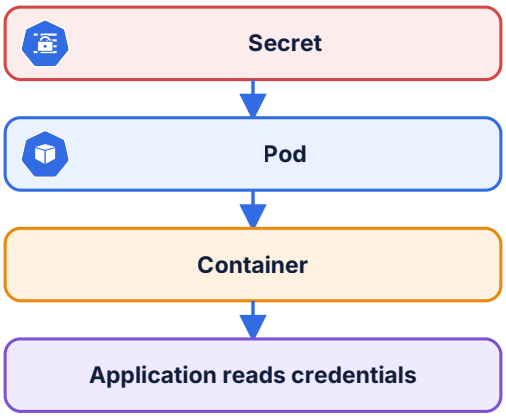



Diagram 11: Secret and Pod

## 9.8 Commands

| Command | What it does |
|---|---|
| `kubectl get secrets` | List Secrets. |
| `kubectl describe secret db-secret` | Show keys and sizes, but not the values. |
| `kubectl get secret db-secret -o yaml` | Show the full Secret. The values are Base64 and easy to decode. |
| `kubectl create secret generic db-secret --from-literal=username=admin --from-literal=password=password123` | Create a Secret without writing YAML. |

## 9.9 Security Best Practices

- **Never commit real passwords to Git**, especially public repositories. Secret YAML files with real values must stay out of version control.
- **RBAC** (Role-Based Access Control) decides who may read which objects. Give only the people and apps that need it permission to read Secrets.
- **Encryption at rest** encrypts Secrets inside etcd, the cluster database. It is not on by default in every setup and must be configured by the cluster administrator (most managed cloud clusters offer it).
- Prefer mounting as files over environment variables; env vars can leak through logs and crash dumps.
- For production, consider an external secrets manager such as HashiCorp Vault or a cloud secrets service.

**COMMON MISTAKES & TROUBLESHOOTING**

- Encoding with `echo` without `-n`, so the password has a hidden newline.
- Putting a plain-text value in `data` instead of `stringData`. Kubernetes rejects invalid Base64.
- Using real passwords in examples, screenshots or Git.
- Pod error `CreateContainerConfigError`: the Secret or key name is wrong.

CHAPTER 10

# Services

## 10.1 What is a Service?

A **Service** gives a group of Pods **one stable address**. Other apps (or users) talk to the Service, and the Service sends the traffic to a healthy Pod.

## 10.2 Why Do We Need Services?

Every Pod has its own IP address, but Pods are temporary. When a Pod is replaced, the new Pod gets a **new IP**. If your front end used the old IP, it would break. A Service address **never changes** while the Service exists.

## 10.3 How a Service Works

- The Service uses a **label selector** to find its Pods (for example `app: nginx`).
- It gets a stable **Service IP** (cluster IP) and a **DNS name** such as `nginx-service.default.svc.cluster.local`. Inside the same Namespace, just `nginx-service` works.
- Traffic to the Service is spread across all **ready** matching Pods.
- Kubernetes keeps the list of Pod addresses in **EndpointSlices**, which update automatically as Pods come and go.

## 10.4 port, targetPort and nodePort

| Field | Meaning |
|---|---|
| **port** | The port the **Service** listens on (inside the cluster). |
| **targetPort** | The port on the **Pod** where traffic is sent. Usually the container port. |
| **nodePort** | The port opened on **every Node** (NodePort type). Range 30000-32767. |

**Example:** `port: 80`, `targetPort: 80`, `nodePort: 30080`. A user visits `NodeIP:30080`. The Node passes it to the Service on port 80. The Service sends it to the Pod on port 80. If the app listened on 8080, you would set `targetPort: 8080` and keep `port: 80`.

## 10.5 The Backend Deployment

All Service examples send traffic to the `nginx-deployment` from Chapter 5 (Pods labelled `app: nginx`). Apply it first:

```
kubectl apply -f deployment.yaml
```

## 10.6 Type A: ClusterIP (default)

Reachable **only inside** the cluster. Used for communication between apps.

service-clusterip.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  type: ClusterIP
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

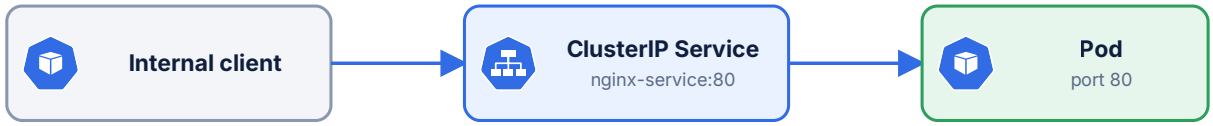



Diagram 12a: ClusterIP

## 10.7 Type B: NodePort

Opens the same port on **every Node**, so outside users can reach the app with `NodeIP:NodePort`. Good for learning and testing.

service-nodeport.yaml

```yaml
apiVersion: v1
kind: Service
metadata:
  name: nginx-nodeport
spec:
  type: NodePort
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
      nodePort: 30080
```

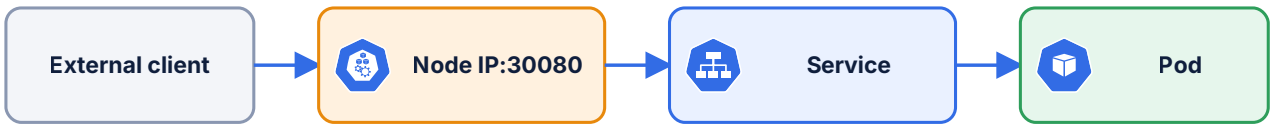



Diagram 12b: NodePort

## 10.8 Type C: LoadBalancer

Asks the **cloud provider** to create an external load balancer with a public address. It includes a NodePort and ClusterIP behind the scenes.

service-loadbalancer.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-lb
spec:
  type: LoadBalancer
  selector:
    app: nginx
  ports:
    - protocol: TCP
      port: 80
      targetPort: 80
```

**NOTE**

LoadBalancer only gets an external IP in environments that support it (AWS, Azure, Google Cloud, or tools such as MetalLB or `minikube tunnel`). Otherwise `EXTERNAL-IP` stays `<pending>`.

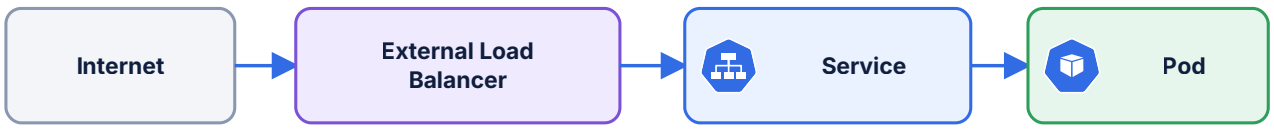



Diagram 12c: LoadBalancer

## 10.9 Type D: ExternalName

Does not select Pods. It makes a Service name point to an **outside DNS name** (a CNAME). Handy for reaching an external database by a friendly in-cluster name.

service-externalname.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: external-db
spec:
  type: ExternalName
  externalName: db.example.com
```




Diagram 12d: ExternalName

## 10.10 Service Types Compared

| Type | Reachable from | Typical use |
|---|---|---|
| **ClusterIP** | Inside the cluster only | Internal communication |
| **NodePort** | Outside, via Node IP and port | Learning, testing |
| **LoadBalancer** | Outside, via a cloud load balancer | Production in cloud |
| **ExternalName** | Inside, redirects to an outside name | Use an external service by name |

## 10.11 Commands

| Command | What it does |
|---|---|
| `kubectl get services` | List Services with type, IP and ports. |
| `kubectl describe service nginx-service` | Show selector, ports and endpoints. |
| `kubectl get endpointslices` | Show the Pod IPs behind each Service. |
| `kubectl get svc -o wide` | Show Services with their selectors. |

**Verify:** if `kubectl describe service` shows `Endpoints: <none>`, the selector does not match any ready Pod.

**COMMON MISTAKES & TROUBLESHOOTING**

- Selector labels do not match the Pod labels. The Service has no endpoints.
- Wrong `targetPort`: the app listens on a different port than the one you sent traffic to.
- Using NodePort in production when a LoadBalancer or Ingress fits better.
- Pods are not ready (failing the readiness probe), so they receive no traffic.

CHAPTER 11

# Ingress

## 11.1 What is Ingress?

**Ingress** is a set of **HTTP and HTTPS routing rules**. It sends web traffic from one entry point to different Services, based on the host name or URL path.

## 11.2 Why Do We Need Ingress?

Without Ingress, each web app would need its own LoadBalancer, which costs money and is hard to manage. With Ingress you use **one entry point** and many rules: `/api` goes to the back end, everything else goes to the front end. Ingress can also handle HTTPS (TLS).

## 11.3 Service vs Ingress

| | Service | Ingress |
|---|---|---|
| **Works at** | Network level (TCP/UDP) | Web level (HTTP/HTTPS) |
| **Routes by** | Labels (selector) | Host names and URL paths |
| **Needs** | Nothing extra | An Ingress Controller |

## 11.4 The Ingress Controller

An Ingress resource is only a **set of rules written down**. Something must read those rules and move the traffic. That program is the **Ingress Controller**, which runs as Pods in the cluster (for example Nginx, Traefik or HAProxy). **Creating an Ingress alone is not enough: without a controller, nothing happens.**

1. You create an Ingress with rules.
2. The controller watches for Ingress objects and reads the rules.
3. It updates its own proxy settings.
4. Web traffic arriving at the controller is sent to the right Service.

> **WARNING**
>
> The popular community **Ingress NGINX** controller was retired in March 2026. It receives no more fixes or security updates. The Ingress API itself still works, and other controllers (Traefik, HAProxy, cloud-provider controllers and more) accept the same kind of YAML. The newer **Gateway API** is the recommended long-term replacement. In this chapter we use `ingressClassName: nginx` as a learning example; set it to match the controller in your own cluster.

## 11.5 A Complete Ingress

First apply the Deployments and Services named `frontend-service` and `backend-service` (both listening on port 80), then apply the Ingress.

ingress.yaml

```yaml
apiVersion: networking.k8s.io/v1
kind: Ingress
metadata:
  name: app-ingress
spec:
  ingressClassName: nginx
  rules:
    - host: app.example.com
      http:
        paths:
          - path: /api
            pathType: Prefix
            backend:
              service:
                name: backend-service
                port:
                  number: 80
          - path: /
            pathType: Prefix
            backend:
              service:
                name: frontend-service
                port:
                  number: 80
```

| Field | Explanation |
| --- | --- |
| **apiVersion: networking.k8s.io/v1** | The current API group and version for Ingress. |
| **kind: Ingress** | The object type. |
| **metadata** | The name (and optional annotations). |
| **spec** | The rules. |
| **ingressClassName: nginx** | Chooses which controller handles this Ingress (an **IngressClass** object names each controller). |
| **rules** | A list of routing rules. |
| **host** | The domain name. If you omit it, the rule matches all hosts. |
| **http / paths** | The list of URL paths and where to send them. |
| **path and pathType** | The URL path to match, and how to match it. |
| **backend: service: name** | The Service that receives the traffic. |
| **port: number** | The **Service port** (not the Pod port). |

## Path types

| pathType | Meaning | Example |
| --- | --- | --- |
| **Prefix** | Matches the path and everything under it, split by `/`. | `/api` matches `/api` and `/api/users`. |

| pathType | Meaning | Example |
|---|---|---|
| **Exact** | Matches only the exact path. | `/api` matches only `/api`. |
| **ImplementationSpecific** | The controller decides how to match. | Behaviour can differ between controllers. |

When several paths match, the **longest** match wins, so `/api` beats `/`.

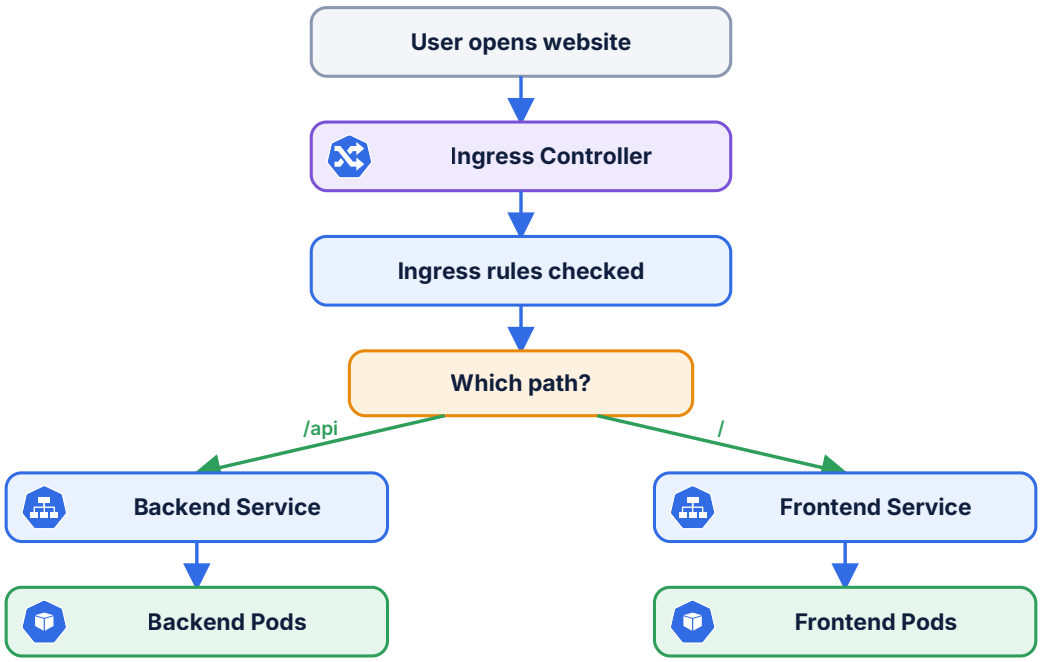



Diagram 13: Ingress path-based routing

## 11.6 Host-Based Routing, TLS and Annotations

- **Host-based routing:** use several `host` rules, such as `shop.example.com` and `blog.example.com`, each pointing to a different Service.
- **TLS (HTTPS):** add a `tls` section that names a TLS Secret (Chapter 9).
- **Annotations:** extra settings for one specific controller, for example `nginx.ingress.kubernetes.io/rewrite-target`. They differ between controllers.

tls snippet

```
spec:
  tls:
    - hosts:
        - app.example.com
      secretName: app-tls
```

## 11.7 NodePort vs LoadBalancer vs Ingress

| | NodePort | LoadBalancer | Ingress |
|---|---|---|---|
| **Protocol** | Any | Any | HTTP/HTTPS |
| **Entry point** | Every Node | One cloud load balancer per Service | One for many Services |

| | NodePort | LoadBalancer | Ingress |
|---|---|---|---|
| **Smart routing (path, host)** | No | No | Yes |
| **Cost** | Low | Higher (one per Service) | Lower (shared) |

## 11.8 Accessing the Application

- Find the controller address: `kubectl get ingress` and `kubectl get svc -n <controller-namespace>`.
- In the cloud, the controller's Service is usually a LoadBalancer with an external IP or hostname.
- On local tools like minikube, you may need a command such as `minikube tunnel` or the Node IP.
- Map the host name to that IP in your `/etc/hosts` file, or test with `curl -H "Host: app.example.com" http://<controller-ip>/api`.

> **NOTE**
> The exact setup depends on your Kubernetes environment (cloud, minikube, kind, or bare metal).

| Command | What it does |
|---|---|
| `kubectl apply -f ingress.yaml` | Create the Ingress. |
| `kubectl get ingress` | Show hosts and the controller address. |
| `kubectl describe ingress app-ingress` | Show rules and events. |
| `kubectl get ingressclass` | List available controllers. |

> **COMMON MISTAKES & TROUBLESHOOTING**
> - No Ingress Controller installed, or the wrong `ingressClassName`.
> - Backend Service name or port is wrong (404 or 503 errors).
> - Host name not mapped to the controller IP.
> - Using the old `extensions/v1beta1` API. It no longer exists.

CHAPTER 12

# Persistent Volumes (PV) and Persistent Volume Claims (PVC)

## 12.1 Why Do Containers Need Storage?

Container storage is **temporary**. Files written inside a container live only as long as that container. When a Pod is deleted or recreated, **that data is lost**. Databases, uploads and logs need storage that **outlives the Pod**.

## 12.2 PersistentVolume (PV) and PersistentVolumeClaim (PVC)

| | PersistentVolume (PV) | PersistentVolumeClaim (PVC) |
|---|---|---|
| **What it is** | A piece of storage available in the cluster. | A request for storage by a user. |
| **Created by** | Administrator, or automatically by a StorageClass. | Developer. |
| **Scope** | Cluster-wide (no Namespace). | Inside a Namespace. |
| **Analogy** | A parking space. | A ticket asking for a parking space. |

> **NOTE**
> A PV is the Kubernetes **object that represents** storage. The actual data is stored on the **underlying storage system**, such as a disk on a Node, a cloud disk or a network share.

## 12.3 How Binding Works

1. A PV exists (or is created) with a size and access mode.
2. You create a PVC that asks for a size, access mode and optional storage class.
3. Kubernetes finds a PV that satisfies the request and **binds** them one-to-one.
4. A Pod uses the PVC as a volume.

The PV must offer at least the requested size. A PVC asking for 500Mi can bind to a 1Gi PV, and the whole PV is then used up by that claim.

## 12.4 Access Modes

| Mode | Short | Meaning |
|---|---|---|
| **ReadWriteOnce** | RWO | Read and write by Pods on **one Node**. |
| **ReadOnlyMany** | ROX | Read-only by Pods on many Nodes. |
| **ReadWriteMany** | RWX | Read and write by Pods on many Nodes (for example NFS). |
| **ReadWriteOncePod** | RWOP | Read and write by **one single Pod** only. |

## 12.5 Volume Types and Reclaim Policies

| Type | Where data lives |
|---|---|
| **hostPath** | A folder on the Node. Only for learning and single-node tests. |
| **AWS EBS** | Cloud block storage disk (used through the EBS CSI driver). |
| **NFS** | A network file share. Supports many Nodes (RWX). |

> **WARNING**
> **hostPath is not suitable for normal production** multi-node clusters. The data stays on one Node, so if the Pod moves to another Node, it will not find its data.

| Reclaim policy | What happens to the PV when its PVC is deleted |
|---|---|
| **Retain** | PV and data are kept. An administrator must clean up before reuse. |
| **Delete** | The PV and the underlying storage are deleted (if the storage type supports it). |
| **Recycle** | Old method that wiped the data. **Deprecated**; do not use it. |

## 12.6 Static Provisioning Example (hostPath)

Apply in this order: PV, then PVC, then Pod.

pv.yaml

```yaml
apiVersion: v1
kind: PersistentVolume
metadata:
  name: nginx-pv
spec:
  capacity:
    storage: 1Gi
  accessModes:
    - ReadWriteOnce
  persistentVolumeReclaimPolicy: Retain
  storageClassName: manual
  hostPath:
    path: /mnt/data
```

pvc.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: nginx-pvc
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: manual
  resources:
    requests:
      storage: 500Mi
```

pod-pvc.yaml

```yaml
apiVersion: v1
kind: Pod
metadata:
  name: nginx-storage-pod
spec:
  containers:
    - name: nginx
      image: nginx:1.25
      ports:
        - containerPort: 80
      volumeMounts:
        - name: web-storage
          mountPath: /usr/share/nginx/html
  volumes:
    - name: web-storage
      persistentVolumeClaim:
        claimName: nginx-pvc
```

| Field | Explanation |
|---|---|
| **capacity.storage** | The size of the PV. |
| **accessModes** | How the volume may be used. |
| **persistentVolumeReclaimPolicy: Retain** | Keep the data after the PVC is deleted. |
| **storageClassName: manual** | A label that lets the PVC and PV match. Here it is just a name; no dynamic provisioning happens. |
| **hostPath.path** | The folder on the Node that holds the data (create it if needed). |
| **PVC resources.requests.storage** | How much storage the claim asks for. |
| **persistentVolumeClaim.claimName** | Connects the Pod to the PVC. |

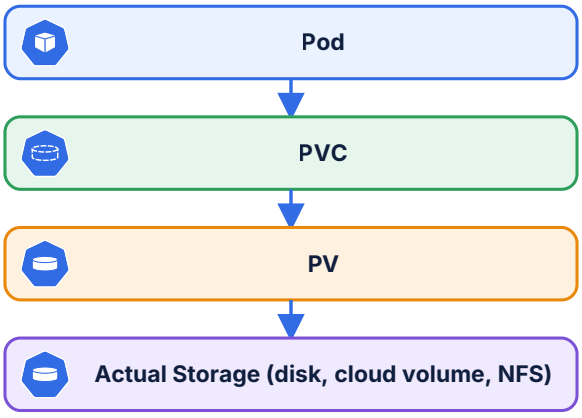



Diagram 14: Pod → PVC → PV → Actual Storage

## 12.7 Lifecycle

- **Available:** the PV is free. **Bound:** the PV is attached to a PVC.
- **Released:** the PVC was deleted, but the PV has not been cleaned for reuse (with Retain).

- **What if a Pod is deleted?** The PVC and data stay. A new Pod can use the same PVC.
- **What if a PVC is deleted?** It depends on the reclaim policy: Retain keeps data, Delete removes it. Kubernetes blocks deleting a PVC while a Pod is still using it.

> **REMEMBER**
> Storage does not automatically mean database replication. A volume only keeps one copy of the data in one place. Copying data between databases must be set up separately.

## 12.8 Commands

| Command | What it does |
|---|---|
| `kubectl get pv` | List PVs and their status (Available, Bound, Released). |
| `kubectl get pvc` | List PVCs and their status (Pending or Bound). |
| `kubectl describe pv` | Show capacity, policy and which claim uses it. |
| `kubectl describe pvc` | Show the bound volume and events. |

**Verify:** both `nginx-pv` and `nginx-pvc` should show `Bound`. Write a file in `/usr/share/nginx/html`, delete the Pod, create it again and check that the file is still there.

> **COMMON MISTAKES & TROUBLESHOOTING**
> - PVC stays `Pending` : no PV matches the size, access mode or `storageClassName` .
> - The hostPath folder does not exist or has wrong permissions.
> - Using hostPath on a multi-node cluster: data seems to disappear when the Pod moves.
> - Deleting a PV by hand while it is still in use.

CHAPTER 13

# StorageClass

## 13.1 What is a StorageClass?

A **StorageClass** describes a **type of storage** that Kubernetes can create on demand (for example "fast SSD disk" or "cheap HDD"). It is like a menu of storage choices.

## 13.2 Why Do We Need It?

In Chapter 12 an administrator had to create each PV by hand. That is slow. With a StorageClass, you only create a PVC and Kubernetes **creates the storage and the PV for you**. This is called **dynamic provisioning**.

| | Static provisioning | Dynamic provisioning |
|---|---|---|
| **Who creates the PV** | Administrator, by hand | Kubernetes, automatically |
| **Needs StorageClass** | No (not required) | Yes |
| **Effort** | High | Low |
| **Typical place** | Learning, special storage | Cloud and modern clusters |

## 13.3 Important Fields

| Field | Meaning |
|---|---|
| **provisioner** | The program (driver) that creates the storage, for example `ebs.csi.aws.com` for AWS EBS. |
| **parameters** | Settings for the provisioner, such as disk type. |
| **reclaimPolicy** | What happens to the storage when the PVC is deleted: `Delete` (default) or `Retain`. |
| **volumeBindingMode** | `Immediate`: create storage right away. `WaitForFirstConsumer`: wait until a Pod uses the PVC, so storage is created in the right place (for example, the right availability zone). |
| **allowVolumeExpansion** | If `true`, you can make a PVC bigger later. |
| **storageClassName (in a PVC)** | Chooses the StorageClass. If you leave it out, the **default** StorageClass is used. |

A **default StorageClass** is marked with the annotation `storageclass.kubernetes.io/is-default-class: "true"`. Check yours with `kubectl get storageclass`; the default shows `(default)`.

## 13.4 Example: AWS EBS

storageclass.yaml

```yaml
apiVersion: storage.k8s.io/v1
kind: StorageClass
metadata:
  name: ebs-sc
provisioner: ebs.csi.aws.com
parameters:
  type: gp3
reclaimPolicy: Delete
volumeBindingMode: WaitForFirstConsumer
allowVolumeExpansion: true
```

pvc-dynamic.yaml

```yaml
apiVersion: v1
kind: PersistentVolumeClaim
metadata:
  name: ebs-claim
spec:
  accessModes:
    - ReadWriteOnce
  storageClassName: ebs-sc
  resources:
    requests:
      storage: 5Gi
```

**WARNING**

This example only works on a cluster that has the **AWS EBS CSI driver installed**, suitable **AWS IAM permissions**, and runs in an appropriate **AWS environment** (for example Amazon EKS). On other clusters, use the StorageClass that your platform provides. With `WaitForFirstConsumer`, the PVC shows `Pending` until a Pod uses it. This is normal.

`gp3` is an EBS SSD disk type. `storageClassName: ebs-sc` links the PVC to this StorageClass.

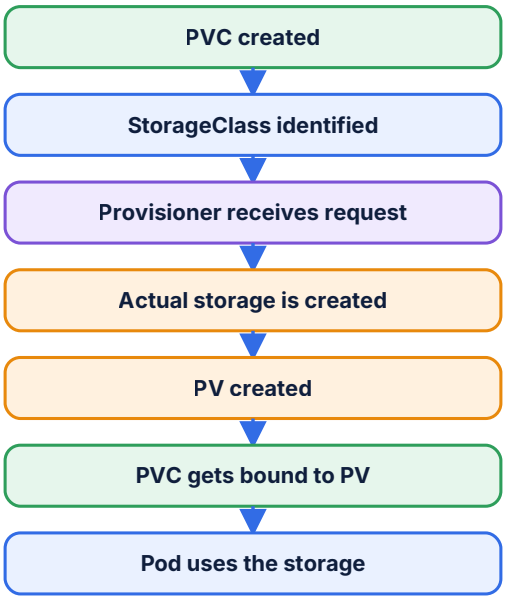



Diagram 15: Dynamic provisioning with a StorageClass

| Command | What it does |
| --- | --- |
| `kubectl get storageclass` | List StorageClasses and see the default. |
| `kubectl describe storageclass ebs-sc` | Show provisioner and settings. |
| `kubectl apply -f pvc-dynamic.yaml` | Create the claim; storage appears automatically. |
| `kubectl get pvc,pv` | See the claim and the PV that was created for it. |

**COMMON MISTAKES & TROUBLESHOOTING**

- No provisioner or CSI driver installed: the PVC stays `Pending`.
- Misspelled `storageClassName`.
- Expecting `WaitForFirstConsumer` PVCs to bind before a Pod exists.
- Forgetting that `reclaimPolicy: Delete` removes the data when the PVC is deleted.

CHAPTER 14

# StatefulSets

## 14.1 What is a StatefulSet?

A **StatefulSet** runs Pods that need a **stable identity** and **their own storage**. It is used for apps that "remember" things, like databases.

## 14.2 Why Do We Need StatefulSets?

Deployment Pods are interchangeable and get random names. A database Pod needs the same name, the same network name and the same disk every time it restarts. StatefulSets give you that.

## 14.3 Deployment vs StatefulSet

| | Deployment | StatefulSet |
|---|---|---|
| **Pod names** | Random ( `web-7d9f-x2k` ) | Ordered ( `mysql-0` , `mysql-1` ) |
| **Network identity** | Not stable | Stable DNS name per Pod |
| **Storage** | Usually shared or none | Separate volume per Pod |
| **Start/stop order** | All at once | In order (by default) |
| **Best for** | Stateless apps | Databases and clustered apps |

## 14.4 Main Features

- **Stable Pod names:** `mysql-0` , `mysql-1` . A replaced Pod keeps its name.
- **Stable network identity:** each Pod gets a DNS name like `mysql-0.mysql.default.svc.cluster.local` , through a **headless Service**.
- **Stable storage:** `volumeClaimTemplates` create one PVC per Pod. The PVC stays attached to that Pod name.
- **Ordered creation:** `mysql-0` must be ready before `mysql-1` starts. **Ordered termination:** Pods stop in reverse order.
- **Parallel management:** set `podManagementPolicy: Parallel` to start and stop Pods all at once (ordering is dropped).

### What is a headless Service?

A Service with `clusterIP: None` . It has no single Service IP. Instead, DNS returns the individual Pod addresses, so you can reach `mysql-0` directly.

## 14.5 Example: MySQL for Learning

> **NOTE**
> This example teaches StatefulSet concepts. It does **not** configure MySQL replication. Create the Secret, then the headless Service, then the StatefulSet. A cluster with a default StorageClass (Chapter 13) is needed for the automatic PVCs.

mysql-secret.yaml

```
apiVersion: v1
kind: Secret
metadata:
  name: mysql-secret
type: Opaque
stringData:
  password: change-me-please
```

mysql-headless-service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: mysql
  labels:
    app: mysql
spec:
  clusterIP: None
  selector:
    app: mysql
  ports:
    - name: mysql
      port: 3306
```

mysql-statefulset.yaml

```yaml
apiVersion: apps/v1
kind: StatefulSet
metadata:
  name: mysql
spec:
  serviceName: mysql
  replicas: 2
  selector:
    matchLabels:
      app: mysql
  template:
    metadata:
      labels:
        app: mysql
    spec:
      containers:
        - name: mysql
          image: mysql:8.0
          ports:
            - containerPort: 3306
              name: mysql
          env:
            - name: MYSQL_ROOT_PASSWORD
              valueFrom:
                secretKeyRef:
                  name: mysql-secret
                  key: password
          readinessProbe:
            tcpSocket:
              port: 3306
            initialDelaySeconds: 20
            periodSeconds: 10
          volumeMounts:
            - name: data
              mountPath: /var/lib/mysql
              subPath: mysql
  volumeClaimTemplates:
    - metadata:
        name: data
      spec:
        accessModes: ["ReadWriteOnce"]
        resources:
          requests:
            storage: 1Gi
```

| Field | Explanation |
| --- | --- |
| **serviceName: mysql** | The headless Service that gives the Pods their DNS names. It **must** exist. |
| **replicas: 2** | Creates `mysql-0` and `mysql-1`. |
| **selector / labels** | Must match, and must match the headless Service selector. |
| **MYSQL_ROOT_PASSWORD from secretKeyRef** | The password comes from the Secret, not from the YAML. |
| **readinessProbe** | Pod is ready only when MySQL accepts connections on port 3306. |
| **subPath: mysql** | Uses a sub-folder of the volume. This avoids MySQL errors caused by the `lost+found` folder that some disks contain. |

| Field | Explanation |
| --- | --- |
| **volumeClaimTemplates** | A template that creates a PVC for each Pod: `data-mysql-0`, `data-mysql-1`. |

> **WARNING**
>
> **Two MySQL Pods are NOT a replicated database.** `mysql-0` and `mysql-1` each have their own PVC, their own PV and their own separate data. If you create a table in `mysql-0`, it will **not** appear in `mysql-1`. Database replication must be set up separately, using MySQL replication tools, an operator, or a purpose-built Helm chart.

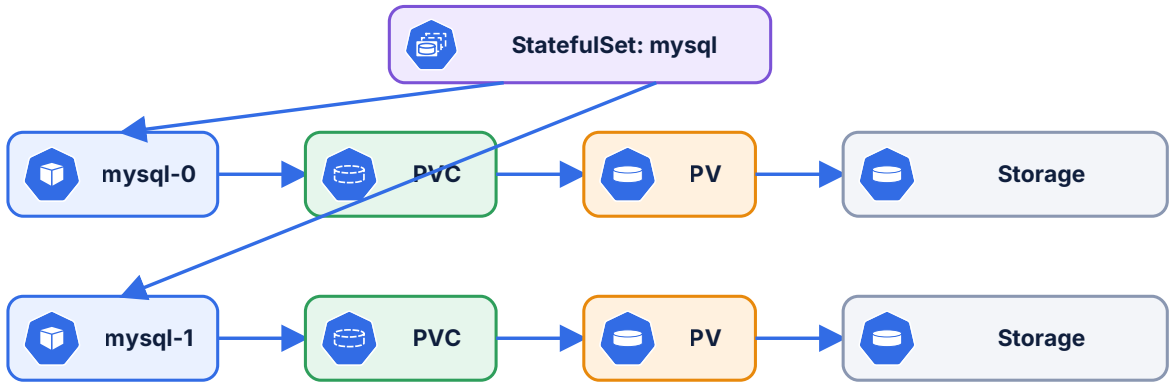



Diagram 16: Each StatefulSet Pod has its own PVC and PV

## 14.6 Scaling, Updates and Limitations

- **Scaling up** adds `mysql-2` with a new PVC. **Scaling down** removes the highest number first. By default the PVCs are **kept**, so data is not lost.
- **Updates** use RollingUpdate by default and replace Pods one at a time in **reverse order** (highest number first).
- **Limitations:** the headless Service must be created by you; deleting the StatefulSet does not delete its PVCs; ordered operations can be slow; a stuck Pod can block later Pods.

**Real-world use cases:** databases (MySQL, PostgreSQL, MongoDB), message queues (Kafka), search engines (Elasticsearch) and coordination tools (ZooKeeper, etcd).

| Command | What it does |
| --- | --- |
| `kubectl get statefulsets` | List StatefulSets. |
| `kubectl get pods` | See `mysql-0` and `mysql-1`. |
| `kubectl get pvc` | See `data-mysql-0` and `data-mysql-1`. |
| `kubectl describe statefulset mysql` | Show details and events. |
| `kubectl scale statefulset mysql --replicas=3` | Add a third Pod, `mysql-2`. |

**COMMON MISTAKES & TROUBLESHOOTING**

- Missing headless Service or wrong `serviceName`.
- No default StorageClass: PVCs stay `Pending` and Pods never start.
- Expecting data to be copied between Pods.
- Putting a real password in a YAML file in Git.

CHAPTER 15

# Helm

## 15.1 What is Helm?

**Helm** is the **package manager for Kubernetes**. Just as `apt` installs software on Ubuntu, Helm installs whole applications on Kubernetes.

## 15.2 Why Do We Need Helm?

A real app needs many YAML files (Deployment, Service, ConfigMap and more). Copying and editing them for every environment is slow and error-prone. Helm packages them together and lets you change settings from one file.

> **ANALOGY**
> Think of a **cake recipe** (the chart) with a list of **choices** (the values): how many people, which flavour. You change the choices, and the same recipe produces a different cake. In Helm, the templates are the recipe and `values.yaml` holds the choices.

## 15.3 Key Terms

| Term | Meaning |
|---|---|
| **Chart** | A package of templates and settings for one application. |
| **Chart.yaml** | The chart's name, description and version. |
| **values.yaml** | Default settings that fill in the templates. |
| **templates/** | YAML files containing placeholders like `{{ .Values.replicaCount }}`. |
| **Release** | One installed copy of a chart in a cluster, with its own name. |
| **Repository** | A place that stores charts, like an app store. |

## 15.4 Chart Structure

```
mychart/
  Chart.yaml
  values.yaml
  templates/
    deployment.yaml
    service.yaml
```

**Chart.yaml**

```yaml
apiVersion: v2
name: mychart
description: A simple Nginx chart
type: application
version: 0.1.0
appVersion: "1.25"
```

**values.yaml**

```yaml
replicaCount: 2
image:
  repository: nginx
  tag: "1.25"
service:
  type: ClusterIP
  port: 80
```

**templates/deployment.yaml**

```yaml
apiVersion: apps/v1
kind: Deployment
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  replicas: {{ .Values.replicaCount }}
  selector:
    matchLabels:
      app: {{ .Release.Name }}-nginx
  template:
    metadata:
      labels:
        app: {{ .Release.Name }}-nginx
    spec:
      containers:
        - name: nginx
          image: "{{ .Values.image.repository }}:{{ .Values.image.tag }}"
          ports:
            - containerPort: 80
```

**templates/service.yaml**

```yaml
apiVersion: v1
kind: Service
metadata:
  name: {{ .Release.Name }}-nginx
spec:
  type: {{ .Values.service.type }}
  selector:
    app: {{ .Release.Name }}-nginx
  ports:
    - port: {{ .Values.service.port }}
      targetPort: 80
```

`{{ .Values.replicaCount }}` is replaced with `2` from `values.yaml`. `{{ .Release.Name }}` is replaced with the release name you choose, so two releases of the same chart do not clash.

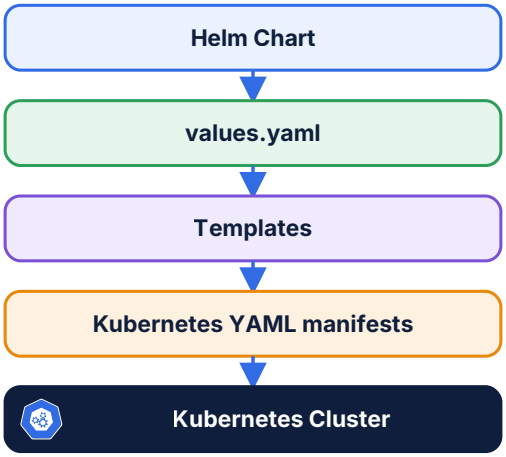



Diagram 17: Helm chart workflow

## 15.5 Commands

| Command | What it does |
|---|---|
| `helm version` | Show the installed Helm version. |
| `helm create mychart` | Generate a starter chart. The default chart deploys Nginx. |
| `helm install my-release ./mychart` | Install the chart as a release named `my-release`. |
| `helm list` | List installed releases. |
| `helm upgrade my-release ./mychart` | Apply changes to an existing release. |
| `helm rollback my-release 1` | Go back to revision 1 of the release. |
| `helm uninstall my-release` | Remove the release and its objects. |
| `helm show values chart-name` | Show the settings a chart accepts. |

### Change values while installing or upgrading

```
helm upgrade my-release ./mychart --set replicaCount=3
helm template my-release ./mychart     # preview the YAML without installing
helm history my-release                # list revisions
```

## 15.6 Install Nginx with Helm (Basic Example)

```
helm create mychart
helm install my-nginx ./mychart
kubectl get pods
helm list
```

The chart made by `helm create` already runs Nginx, so it is the easiest first example. To use a chart from a public repository, the pattern is:

```
helm repo add <repo-name> <repo-url>
helm repo update
helm search repo nginx
helm install my-nginx <repo-name>/nginx
```

> **NOTE**
> Public chart catalogs change over time. For example, the free Bitnami chart images were reorganised in 2025, so some older commands now need image overrides. Use **artifacthub.io** to find a chart that is currently maintained, and read its page before installing.

## 15.7 Upgrade and Rollback

Every install or upgrade creates a numbered **revision**. If an upgrade goes wrong, `helm rollback my-release 1` restores revision 1. Helm tracks revisions itself; this is separate from Deployment rollouts.

> **COMMON MISTAKES & TROUBLESHOOTING**
> - Indentation mistakes inside templates.
> - Using a value in a template that is missing from `values.yaml`.
> - Forgetting the release name in `helm install`.
> - Not previewing with `helm template` before installing.

CHAPTER 16

# Kustomize

## 16.1 What is Kustomize?

**Kustomize** customises plain Kubernetes YAML **without templates**. You keep one common set of files and add small changes for each environment. It is built into `kubectl`.

## 16.2 Why Do We Need Kustomize?

Development, testing and production are almost the same, but small things differ: replica count, image version, names. Copying whole YAML files for each environment is messy. Kustomize keeps one **base** and applies small **overlays**.

| Term | Meaning |
| --- | --- |
| **kustomization.yaml** | The file that lists resources and the changes to apply. |
| **Base** | The common YAML shared by all environments. |
| **Overlay** | A folder that points to the base and changes parts of it for one environment. |

## 16.3 Project Structure

```
myapp/
  base/
    deployment.yaml
    service.yaml
    kustomization.yaml
  overlays/
    development/
      kustomization.yaml
    production/
      kustomization.yaml
```

## 16.4 The Base

base/deployment.yaml

```
apiVersion: apps/v1
kind: Deployment
metadata:
  name: nginx-deployment
spec:
  replicas: 2
  selector:
    matchLabels:
      app: nginx
  template:
    metadata:
      labels:
        app: nginx
    spec:
      containers:
        - name: nginx
          image: nginx
          ports:
            - containerPort: 80
```

base/service.yaml

```
apiVersion: v1
kind: Service
metadata:
  name: nginx-service
spec:
  selector:
    app: nginx
  ports:
    - port: 80
      targetPort: 80
```

base/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - deployment.yaml
  - service.yaml
```

The base image is just `nginx` with no tag. Each overlay chooses the tag. `resources` lists the files Kustomize should include.

## 16.5 The Overlays

overlays/development/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namePrefix: dev-
replicas:
  - name: nginx-deployment
    count: 1
images:
  - name: nginx
    newTag: 1.25-alpine
```

overlays/production/kustomization.yaml

```
apiVersion: kustomize.config.k8s.io/v1beta1
kind: Kustomization
resources:
  - ../../base
namePrefix: prod-
replicas:
  - name: nginx-deployment
    count: 5
images:
  - name: nginx
    newTag: "1.26"
```

| Field | Explanation |
| --- | --- |
| **resources: ../../base** | Start from the base files. |
| **namePrefix** | Adds a prefix to object names ( `dev-nginx-deployment` ), so both environments can run side by side. |
| **replicas** | Changes the replica count of the named Deployment. |
| **images: newTag** | Changes the tag of the image named `nginx` . You can also use `newName` to change the image name. |

The development overlay gives **1 replica** and a development image ( `nginx:1.25-alpine` ). The production overlay gives **5 replicas** and a production image ( `nginx:1.26` ). Kustomize creates the final YAML from base + overlay, so you never maintain separate full copies.

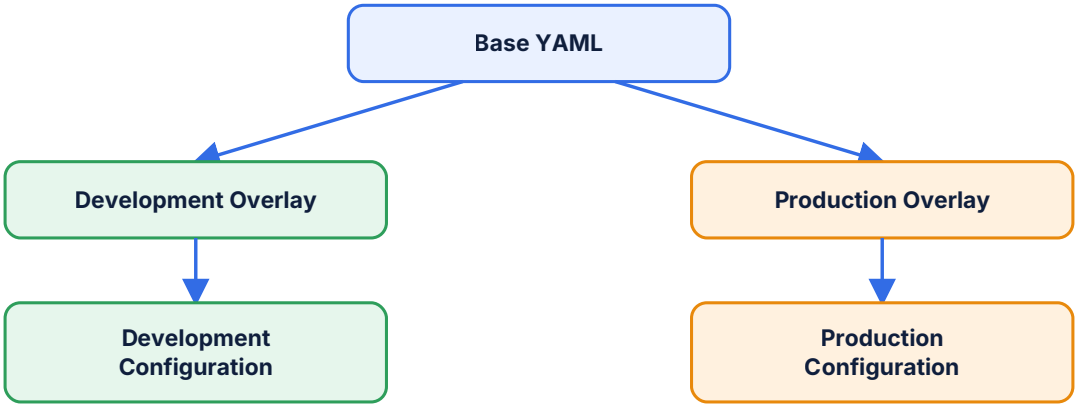



Diagram 18: Kustomize base and overlays

## 16.6 Commands

| Command | What it does |
|---|---|
| `kubectl kustomize ./base` | Print the YAML that the base produces. |
| `kubectl kustomize ./overlays/development` | Preview the final development YAML without applying it. |
| `kubectl apply -k ./overlays/development` | Build and apply the development overlay. |
| `kubectl apply -k ./overlays/production` | Build and apply the production overlay. |

### `kubectl apply -f` vs `kubectl apply -k`

| | apply -f | apply -k |
|---|---|---|
| **Input** | A file, a folder of files, or a URL | A folder that contains `kustomization.yaml` |
| **What it does** | Applies the YAML exactly as written | Builds the final YAML first (base + changes), then applies it |
| **Use for** | Simple, single manifests | Environment-specific variations |

## 16.7 Helm vs Kustomize

| | Helm | Kustomize |
|---|---|---|
| **Approach** | Templates with placeholders | Plain YAML with patches and overlays |
| **Packaging and versions** | Charts, releases, rollback | No packaging, just customising |
| **Learning curve** | Higher (template language) | Lower |
| **Built into kubectl** | No (separate tool) | Yes |
| **Best for** | Installing shared applications | Managing your own app across environments |

**COMMON MISTAKES & TROUBLESHOOTING**

- Wrong relative path in `resources`.
- The `name` in `images` or `replicas` does not match the base.
- Running `kubectl apply -f` on an overlay folder (use `-k`).
- Forgetting that `namePrefix` changes object names.

CHAPTER 17

# Quick Comparison and Revision

Use these tables for quick revision. Each one answers one simple question.

## 17.1 Pod vs ReplicaSet vs Deployment

| | Pod | ReplicaSet | Deployment |
|---|---|---|---|
| **Purpose** | Runs containers | Keeps N Pods running | Manages ReplicaSets and updates |
| **Self-healing** | No | Yes | Yes |
| **Rolling update and rollback** | No | No | Yes |
| **Use directly?** | Rarely | Rarely | Yes, for stateless apps |

## 17.2 Deployment vs DaemonSet vs StatefulSet

| | Deployment | DaemonSet | StatefulSet |
|---|---|---|---|
| **Number of Pods** | You choose | One per eligible Node | You choose |
| **Pod names** | Random | Random | Ordered (name-0, name-1) |
| **Storage** | Shared or none | Usually Node files | Own volume per Pod |
| **Typical use** | Web apps, APIs | Log and monitoring agents | Databases |

## 17.3 ConfigMap vs Secret

| | ConfigMap | Secret |
|---|---|---|
| **Data** | Normal settings | Passwords, tokens, keys |
| **Stored as** | Plain text | Base64 (not encryption) |
| **Can use as** | Env vars, files | Env vars, files |
| **Protect with** | Normal access control | RBAC, encryption at rest |

## 17.4 ClusterIP vs NodePort vs LoadBalancer vs Ingress

| | ClusterIP | NodePort | LoadBalancer | Ingress |
|---|---|---|---|---|
| **Reachable from** | Inside cluster | Outside, via Node port | Outside, via cloud LB | Outside, via controller |
| **Protocol** | Any | Any | Any | HTTP/HTTPS |
| **Path/host routing** | No | No | No | Yes |
| **Needs** | Nothing | Open Node ports | Cloud support | Ingress Controller |

## 17.5 PV vs PVC vs StorageClass

| | PV | PVC | StorageClass |
|---|---|---|---|
| **What it is** | The storage | A request for storage | A type of storage |
| **Created by** | Admin or StorageClass | Developer | Admin |
| **Namespaced?** | No | Yes | No |

| | PV | PVC | StorageClass |
|---|---|---|---|
| **Role** | Represents real storage | Asks for storage | Creates PVs on demand |

## 17.6 Deployment vs StatefulSet

| | Deployment | StatefulSet |
|---|---|---|
| **Pod identity** | Random, replaceable | Fixed name and DNS |
| **Storage** | Same for all or none | One PVC per Pod |
| **Start order** | All together | One by one (default) |
| **Needs headless Service** | No | Yes (recommended) |

## 17.7 Helm vs Kustomize

| | Helm | Kustomize |
|---|---|---|
| **Method** | Templates and values | Base and overlays |
| **Versioned releases** | Yes | No |
| **Built into kubectl** | No | Yes |
| **Best for** | Installing shared apps | Your own app in many environments |

## 17.8 Static vs Dynamic Provisioning

| | Static | Dynamic |
|---|---|---|
| **PV created by** | Administrator | Provisioner, automatically |
| **Needs StorageClass** | No | Yes |
| **Speed** | Slow | Fast |
| **Common in** | Learning, special storage | Cloud clusters |

## 17.9 Service vs Ingress

| | Service | Ingress |
|---|---|---|
| **Level** | Network (TCP/UDP) | Web (HTTP/HTTPS) |
| **Routing by** | Label selector | Host and path |
| **Extra software** | No | Ingress Controller |
| **Use for** | Stable access to Pods | Smart routing, one entry point, TLS |

CHAPTER 18

# Command Quick Reference

A short list of the most useful commands, grouped by topic.

## Namespaces

| Command | What it does |
| --- | --- |
| `kubectl get namespaces` | List Namespaces. |
| `kubectl create namespace NAME` | Create a Namespace. |
| `kubectl delete namespace NAME` | Delete it and everything inside. |
| `kubectl config set-context --current --namespace=NAME` | Change the default Namespace. |

## Pods

| Command | What it does |
| --- | --- |
| `kubectl apply -f pod.yaml` | Create a Pod. |
| `kubectl get pods -o wide` | List Pods with IP and Node. |
| `kubectl describe pod NAME` | Show details and events. |
| `kubectl logs NAME` | Show container logs. |
| `kubectl exec -it NAME -- sh` | Open a shell in a Pod. |
| `kubectl delete pod NAME` | Delete a Pod. |

## ReplicaSets

| Command | What it does |
| --- | --- |
| `kubectl get replicasets` | List ReplicaSets. |
| `kubectl describe replicaset NAME` | Show details. |
| `kubectl scale replicaset NAME --replicas=N` | Change the number of Pods. |

## Deployments

| Command | What it does |
| --- | --- |
| `kubectl apply -f deployment.yaml` | Create or update. |
| `kubectl get deployments` | List Deployments. |
| `kubectl describe deployment NAME` | Show details. |

| Command | What it does |
| --- | --- |
| `kubectl scale deployment NAME --replicas=N` | Scale. |
| `kubectl delete deployment NAME` | Delete. |

## Rolling Updates

| Command | What it does |
| --- | --- |
| `kubectl set image deployment/NAME CONTAINER=IMAGE` | Change the image. |
| `kubectl rollout status deployment/NAME` | Watch the rollout. |
| `kubectl rollout history deployment/NAME` | List revisions. |
| `kubectl rollout undo deployment/NAME` | Roll back to the previous revision. |
| `kubectl rollout restart deployment/NAME` | Restart Pods gradually. |
| `kubectl rollout pause deployment/NAME` | Pause a rollout. |
| `kubectl rollout resume deployment/NAME` | Resume a rollout. |

## DaemonSets

| Command | What it does |
| --- | --- |
| `kubectl get daemonsets` | List DaemonSets. |
| `kubectl describe daemonset NAME` | Show details. |

## ConfigMaps

| Command | What it does |
| --- | --- |
| `kubectl get configmaps` | List ConfigMaps. |
| `kubectl describe configmap NAME` | Show contents. |
| `kubectl create configmap NAME --from-literal=KEY=VALUE` | Create from a value. |
| `kubectl create configmap NAME --from-file=FILE` | Create from a file. |

## Secrets

| Command | What it does |
| --- | --- |
| `kubectl get secrets` | List Secrets. |
| `kubectl describe secret NAME` | Show keys (not values). |
| `kubectl get secret NAME -o yaml` | Show the Base64 values. |
| `kubectl create secret generic NAME --from-literal=KEY=VALUE` | Create a Secret. |

## Services

| Command | What it does |
| --- | --- |
| `kubectl get svc -o wide` | List Services with selectors. |
| `kubectl describe service NAME` | Show ports and endpoints. |
| `kubectl get endpointslices` | Show Pod addresses behind Services. |

## Ingress

| Command | What it does |
| --- | --- |
| `kubectl get ingress` | List Ingress rules and addresses. |
| `kubectl describe ingress NAME` | Show rules and events. |
| `kubectl get ingressclass` | List controllers. |

## PV, PVC and StorageClass

| Command | What it does |
| --- | --- |
| `kubectl get pv` | List PersistentVolumes. |
| `kubectl get pvc` | List claims. |
| `kubectl describe pvc NAME` | Show binding and events. |
| `kubectl get storageclass` | List StorageClasses. |

## StatefulSets

| Command | What it does |
| --- | --- |
| `kubectl get statefulsets` | List StatefulSets. |
| `kubectl describe statefulset NAME` | Show details. |
| `kubectl scale statefulset NAME --replicas=N` | Scale. |

## Helm

| Command | What it does |
| --- | --- |
| `helm create NAME` | Create a starter chart. |
| `helm install RELEASE CHART` | Install a chart. |
| `helm list` | List releases. |
| `helm upgrade RELEASE CHART` | Upgrade a release. |
| `helm history RELEASE` | List revisions. |

| Command | What it does |
| --- | --- |
| `helm rollback RELEASE N` | Roll back to revision N. |
| `helm uninstall RELEASE` | Remove a release. |
| `helm template RELEASE CHART` | Preview generated YAML. |
| `helm show values CHART` | Show default settings. |

## Kustomize

| Command | What it does |
| --- | --- |
| `kubectl kustomize DIR` | Print the final YAML. |
| `kubectl apply -k DIR` | Build and apply. |
| `kubectl delete -k DIR` | Delete what the kustomization created. |